

Análise e Desenvolvimento de Sistemas

André Oliva Caliento

Diciplina: Desenvolvimento em javascript
Unidade 2: Bibliotecas para desenvolvimento em javascript
Aula 2: APIS de navegador manipulando áudio e gráfico

André Oliva Caliento

Diciplina: Desenvolvimento em javascript
Unidade 2: Bibliotecas para desenvolvimento em javascript
Aula 2: APIS de navegador manipulando áudio e gráfico

Sumario

1- Introdução.....	4
2- Desenvolvimento.....	5
3- Prints da atividade.....	8
4- Conclusão.....	13
5- Referencia.....	14

Introdução

A atividade proposta tem como objetivo explorar o uso do elemento canvas do HTML5, aliado à linguagem JavaScript, para a criação de animações gráficas em páginas web. O canvas é uma ferramenta poderosa que permite desenhar formas, imagens e criar movimentos dinâmicos diretamente no navegador, sem a necessidade de plugins externos. Ao desenvolver este experimento, o estudante tem contato com conceitos fundamentais de programação gráfica e interatividade, aplicando lógica de programação para transformar páginas estáticas em ambientes visuais dinâmicos.

Desenvolvimento

O trabalho foi dividido em etapas práticas:

1. **Configuração do projeto**
 - Criação de um arquivo HTML contendo o elemento `<canvas>`.
 - Definição de largura e altura para delimitar a área de desenho.
 - Conexão com um arquivo JavaScript responsável pela lógica da animação.
2. **Configuração inicial**
 - Obtenção do contexto 2D do canvas (`getContext("2d")`), que funciona como o “pincel” para desenhar.
3. **Desenho de objetos**
 - Implementação de funções para desenhar círculos usando `ctx.arc()`.
 - Definição de cores e preenchimento com `ctx.fillStyle` e `ctx.fill()`.
4. **Animação básica**
 - Uso do `requestAnimationFrame` para criar um loop contínuo de atualização.
 - Alteração das propriedades dos objetos (posição, cor) para gerar movimento.
 - Implementação de lógica para rebater nas bordas do canvas.
5. **Interatividade opcional**
 - Adição de eventos de clique para mudar a cor do círculo, tornando a animação interativa.
6. **Estruturas de repetição**
 - Reflexão sobre o uso do laço `for` em sistemas interativos, como o exemplo da tabuada.

Loops permitem desenhar múltiplos objetos ou calcular várias operações sem código repetitivo, tornando o sistema mais eficiente e escalável.

Código HTML

Descrição: Este trecho cria a estrutura da página web. O elemento `<canvas>` define a área gráfica onde os objetos serão desenhados. O arquivo `script_circulo.js` contém a lógica da animação.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Animação Canvas</title>
</head>
<body>
  <!-- O elemento canvas é onde desenharemos -->
  <canvas id="meuCanvas" width="500" height="400" style="border:1px solid
black;"></canvas>

  <!-- Ligação com o arquivo JavaScript -->
  <script src="script_circulo.js"></script>
</body>
</html>
```

Código JavaScript

Descrição:

- O código obtém o contexto 2D do canvas.
- Cria um objeto círculo com propriedades como posição, raio, cor e velocidade.
- A função `desenharCirculo()` desenha o círculo.
- A função `animar()` atualiza a posição e cria o movimento contínuo com `requestAnimationFrame`.
- O evento de clique permite mudar a cor do círculo, tornando a animação interativa.

```
• // 1. Obter o canvas e o contexto 2D
• const canvas = document.getElementById("meuCanvas");
• const ctx = canvas.getContext("2d");
•
• // 2. Criar um objeto círculo com propriedades
• let circulo = {
•   x: 50,           // posição inicial no eixo X
•   y: 200,          // posição inicial no eixo Y
•   raio: 20,        // tamanho do círculo
•   cor: "blue",     // cor inicial
•   dx: 2            // velocidade horizontal
• };
•
• // 3. Função para desenhar o círculo
• function desenharCirculo() {
•   ctx.beginPath(); // inicia o desenho
•   ctx.arc(circulo.x, circulo.y, circulo.raio, 0, Math.PI * 2);
•   ctx.fillStyle = circulo.cor;
•   ctx.fill();
•   ctx.closePath(); // finaliza o desenho
• }
•
• // 4. Função de animação
• function animar() {
•   // Limpa o canvas a cada frame
•   ctx.clearRect(0, 0, canvas.width, canvas.height);
•
•   // Desenha o círculo
•   desenharCirculo();
•
•   // Atualiza a posição
•   circulo.x += circulo.dx;
•
•   // Faz o círculo "rebater" nas bordas
•   if (circulo.x > canvas.width - circulo.raio || circulo.x <
circulo.raio) {
•     circulo.dx *= -1; // inverte a direção
•   }
• }
```

```
•   }  
•  
•   // Chama o próximo frame  
•   requestAnimationFrame(animar);  
•   }  
•  
•   // 5. Iniciar a animação  
•   animar();  
•  
•   // 6. Interatividade opcional: mudar cor ao clicar  
•   canvas.addEventListener("click", () => {  
•     circulo.cor = circulo.cor === "blue" ? "red" : "blue";  
•   });  
•
```


Prints da atividade

https://github.com/caliento313/DESENVOLVIMENTO_EM_JAVASCRIPT

Descrição da Atividade

A tela exibida no navegador apresenta um elemento <canvas> delimitado por uma borda preta, dentro do qual foi desenhado um círculo azul. Esse círculo é o objeto gráfico definido no código JavaScript e está sendo renderizado dinamicamente no canvas.

O projeto demonstra:

- A configuração do canvas no HTML, que cria a área gráfica.
- O uso do contexto 2D para desenhar formas geométricas.
- A aplicação de funções em JavaScript para desenhar objetos e controlar suas propriedades (posição, raio, cor).
- A lógica de animação com `requestAnimationFrame`, que permite atualizar continuamente a posição do círculo, criando movimento.
- A possibilidade de interatividade, já que o código também permite alterar a cor do círculo ao clicar sobre o canvas.

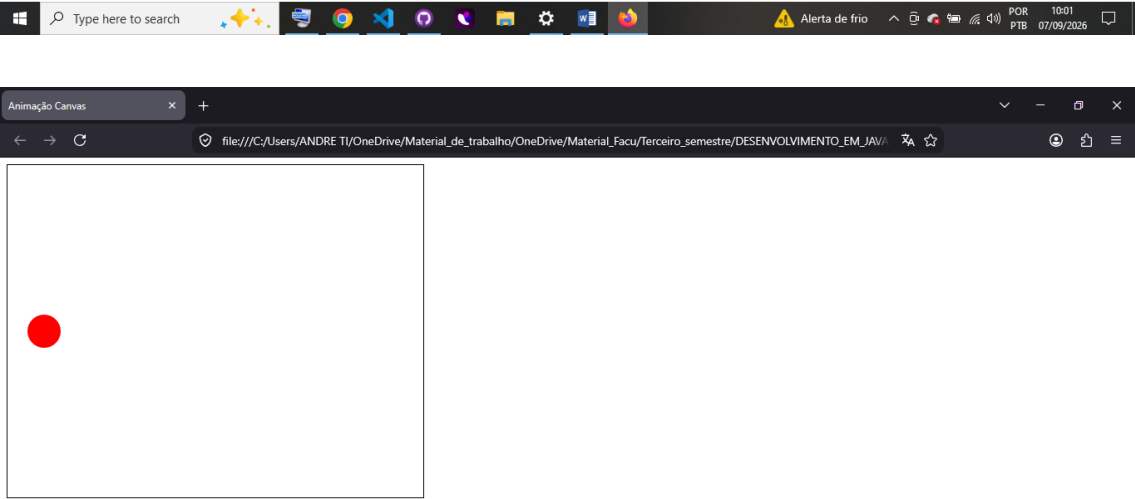
Interpretação Didática

O print evidencia que o código foi corretamente implementado:

- O círculo azul aparece dentro da área do canvas, confirmando que o contexto gráfico está ativo.
- O posicionamento centralizado mostra que as coordenadas foram aplicadas corretamente.
- O projeto cumpre o objetivo da atividade: transformar uma página web simples em uma experiência visual e interativa, utilizando apenas HTML5 e JavaScript.

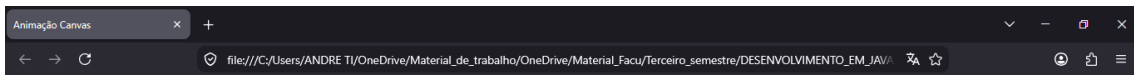


Activate Windows
Go to Settings to activate Windows.

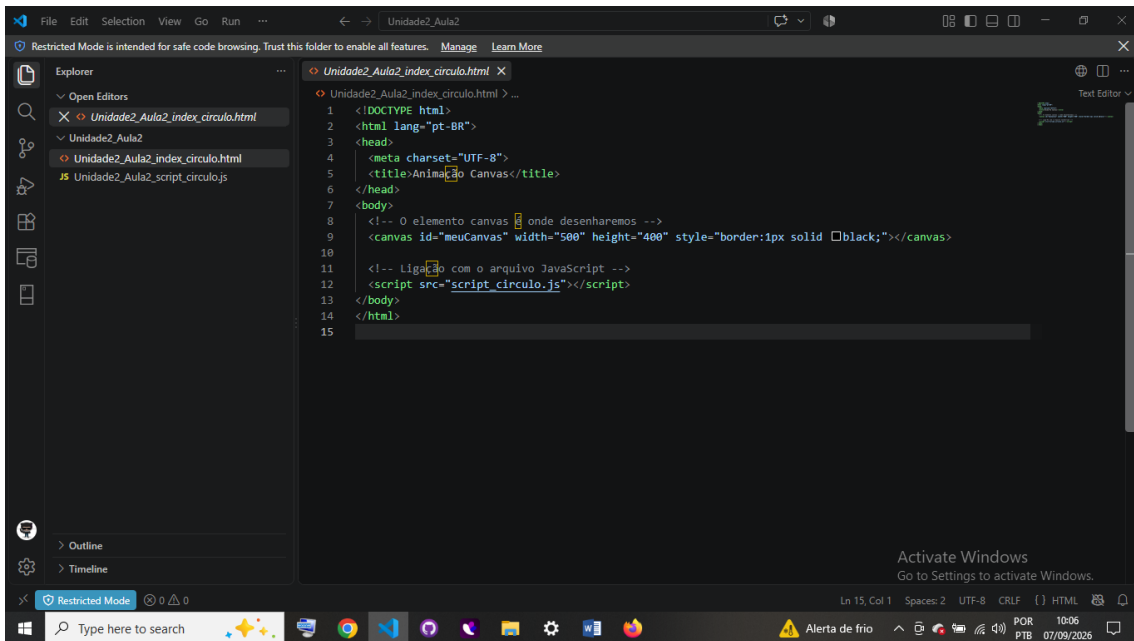


Activate Windows
Go to Settings to activate Windows.





Activate Windows
Go to Settings to activate Windows.



Activate Windows
Go to Settings to activate Windows.

The screenshot shows the Visual Studio Code editor with a file named `Unidade2_Aula2_script_circulo.js` open. The code is written in JavaScript and includes comments in Portuguese. It defines a canvas, a context, and a circle object with initial properties. It also defines a function to draw the circle and a function to animate it.

```
1 // 1. Obter o canvas e o contexto 2D
2 const canvas = document.getElementById("meuCanvas");
3 const ctx = canvas.getContext("2d");
4
5 // 2. Criar um objeto círculo com propriedades
6 let circulo = {
7   x: 50,      // posição inicial no eixo X
8   y: 200,     // posição inicial no eixo Y
9   raio: 20,   // tamanho do círculo
10  cor: "blue", // cor inicial
11  dx: 2        // velocidade horizontal
12};
13
14 // 3. Função para desenhar o círculo
15 function desenharCirculo() {
16   ctx.beginPath(); // inicia o desenho
17   ctx.arc(circulo.x, circulo.y, circulo.raio, 0, Math.PI * 2);
18   ctx.fillStyle = circulo.cor;
19   ctx.fill();
20   ctx.closePath(); // finaliza o desenho
21}
22
23 // 4. Função de animação
24 function animar() {
25   // Limpa o canvas a cada frame
26   ctx.clearRect(0, 0, canvas.width, canvas.height);
27
28   // Desenha o círculo
29   desenharCirculo();
30}
31
32 // Animação a cada frame
```

The screenshot shows the Visual Studio Code editor with the same file `Unidade2_Aula2_script_circulo.js` open. The code continues from the previous screenshot, defining the `animar` function to update the circle's position and handle boundary collisions. It also includes an optional click event listener to change the circle's color.

```
24 function animar() {
25   // Desenha o círculo
26   desenharCirculo();
27
28   // Atualiza a posição
29   circulo.x += circulo.dx;
30
31   // Faz o círculo "rebater" nas bordas
32   if (circulo.x > canvas.width - circulo.raio || circulo.x < circulo.raio) {
33     circulo.dx *= -1; // inverte a direção
34   }
35
36   // Chama o próximo frame
37   requestAnimationFrame(animar);
38}
39
40 // 5. Iniciar a animação
41 animar();
42
43 // 6. Interatividade opcional: mudar cor ao clicar
44 canvas.addEventListener("click", () => {
45   circulo.cor = circulo.cor === "blue" ? "red" : "blue";
46 });
47
48 // Animação a cada frame
```

Conclusão

A realização desta atividade evidenciou a importância do canvas como recurso para o desenvolvimento de animações e sistemas interativos em páginas web. O processo de implementação reforçou conceitos essenciais de programação, como manipulação de objetos, uso de funções, controle de fluxo e interatividade com eventos. Além disso, destacou o papel das estruturas de repetição na otimização de tarefas, permitindo que o código seja mais organizado e capaz de lidar com múltiplos elementos simultaneamente.

Assim, o trabalho não apenas introduziu o uso prático do canvas, mas também demonstrou como a lógica de programação aplicada ao JavaScript pode transformar páginas simples em experiências visuais dinâmicas, criativas e envolventes, aproximando o estudante das práticas utilizadas em jogos, simulações e aplicações modernas.

Referências

ALVES, W. P. **HTML e CSS**: aprenda como construir páginas web. São Paulo: Expressa, 2021.

FLANAGAN, D. **JavaScript**: o guia definitivo. Tradução: João Eduardo Nóbrega Tortello. 6. ed. Porto Alegre: Bookman, 2013.

MDN WEB DOCS. **Canvas**. 2024. Disponível em: [https://developer.mozilla.org/pt-BR/docs/Web/API/Canvas API](https://developer.mozilla.org/pt-BR/docs/Web/API/Canvas_API). Acesso em: 11 jan. 2024.